

LLM Flight Assistant for UAV Drones — Research & Architecture Report (v2)

Scope: PX4 autopilot + MAVLink, SITL-first (Gazebo/jMAVSim), later HITL. LLM serves four roles:

(1) natural-language command & control, (2) mission planning/replanning, (3) telemetry & log analysis/diagnostics, (4) safety/operator copilot. **Project context: likely built for / aligned with Baykar and a TEKNOFEST entry — see §10.**

Method: Two research passes. Pass 1: multi-source web research with 3-vote adversarial fact-checking (26 sources). Pass 2 (this revision): five parallel deep-dives reading primary sources end-to-end (papers, the actual code repos, dataset cards, competition rules, Turkish-language industry sources). Headline metrics below are quoted from the source tables, not paraphrased. Date: 2026-06-08.

Confidence legend: ✓ verified against primary source · ⚠ single-source / caveated · ⚙ engineering judgment (not researched fact). **v2 changelog:** corrected the 40%/100% metric conflation; added LeRAAT, TypeFly/MiniSpec, the droneserver MCP code, a datasets section, a landscape/survey section, and a Baykar/TEKNOFEST positioning section.

0. Executive summary

A PX4+MAVLink+SITL flight assistant is **feasible today** and is fundamentally an **integration + safety-engineering project**, not an unsolved-research moonshot. The non-negotiable rule, agreed by every serious source: **the LLM never sits in the hard real-time control loop**. It reasons over a high-level, descriptively-named tool/function library wrapping the real flight stack (MAVSDK/pymavlink/ROS 2); a deterministic validation gate checks every command; a human stays on the loop.

The reality check, stated correctly this time: the closest open PX4 system gets ****100% valid command generation from small local models (Gemma3/Qwen2.5/Llama-3.2) but only ~40% *end-to-end mission success***** — and those are different metrics (§1). Validity is easy; *correct values + perception + multi-step completion* is the hard part. Plan your effort around closing that gap (telemetry-gated execution, grounding, verification) — not around whether the LLM can emit a syntactically valid command.

The two closest published blueprints are **LeRAAT** (RAG aviation-advisory over X-Plane — the advisory half, but with essentially no quantitative validation) and **TypeFly** (a constrained mini-language that cuts latency up to 62% and tokens ~34% — the command half, well-measured on real hardware). Combine LeRAAT's RAG/advisory spine with TypeFly's compact-DSL command layer, behind a safety validator.

Where a new project genuinely differentiates: telemetry/log analysis and a **regulatory-grounded safety copilot** are under-served, and — given the Baykar/TEKNOFEST context — a **Turkish-language, RAG-grounded operator assistant** (potentially built on/benchmarked against the Turkish T3 AI model) maps directly onto both what Baykar publicly recruits for and the TEKNOFEST Turkish-NLP competition (§10).

1. State of the art / prior work

1.1 Foundational robotics-LLM precedents (design patterns that don't expire)

Work	What it demonstrated	Relevance
Microsoft "ChatGPT for Robotics" / PromptCraft-Robotics (2023) ✓	Drove a simulated AirSim drone and a real DJI Tello from natural language + a base API list (circular/lawnmower inspection patterns).	The canonical high-level function-library pattern + human-on-the-loop. C2 blueprint.
SayCan (Google, 2022) ✓	Grounds the LLM by multiplying LLM "usefulness" × a value-function "is it feasible now" affordance , picking the max-product skill.	The grounding pattern: LLM proposes, a feasibility layer (geofence/battery/airspace/sensor state) gates.
Code as Policies (Google, ICRA 2023) ✓	Code-writing LLMs generate policy code — functions/feedback loops over perception that parameterize control-primitive APIs.	Justifies exposing the flight stack as a clean API the LLM composes.

1.2 Direct UAV / PX4 precedents (the closest matches)

- **Tazir et al., "From Words to Flight"** (WCSE 2023) ✓ — GPT-3.5-Turbo + **PX4/Gazebo** via a Python middleware chatbot converting natural language → MAVLink commands. The single most on-target prior work for *your exact stack*; cite as direct precedent.
- **"Taking Flight with Dialogue"** (Lim et al., arXiv **2506.07509**, 2025) ✓ — open-source **PX4 + ROS 2 + Ollama (local models)** agentic framework; code github.com/limshoonkit/ros2-agent-ws. **See the metric box below — its numbers are widely mis-cited.** Action space was **forward/yaw only at ~1 m altitude** (not 3D); simulator was **Isaac Sim** (not Gazebo); hardware = custom quad + Jetson Orin Nano + ZED Mini + Pixhawk 6c Mini.
- **LeRAAT** (Stanford SISL, arXiv **2503.16477**) ✓ — **the closest published blueprint for an advisory tool.** Detailed in §1.4.
- **TypeFly** (Yale, arXiv **2312.14950**; IEEE TMM 2025) ✓ — **the closest published blueprint for a low-latency command layer.** Detailed in §1.5.
- **MAVLink-AI-Agent** (github.com/SuperMK15/MAVLink-AI-Agent) ⚠ — student PoC proving **voice → LLM (Cohere) + light RAG → pymavlink execution** end-to-end. Single commit, 2 stars, Windows-targeted TTS, 7-command vocabulary. Useful as an honest **MVP-scope gauge** and voice-interface reference, not a dependency.

✓ *The metric that everyone (including my v1) gets wrong — corrected* In "Taking Flight with Dialogue" (2506.07509), Table I reports **two distinct columns**: - **Valid flight-command generation** (output is a well-formed Turn/Move within allowed ranges — explicitly NOT whether the value is correct): **Gemma3 4B = 100%, Qwen2.5 3B = 100%, Llama-3.2 3B = 100%, DeepSeek-LLM 7B = 38%**. - **Mission success rate** (locate + approach target within 0.5 m — a combined LLM+VLM end-to-end metric, 20 episodes): **best 40%** (Gemma3 LLM + Gemma3 12B VLM); most pairings 30–35%; DeepSeek 0–5%. The authors: "even in scenarios where valid commands were generated and object presence was accurately detected at 100%, some mission failures occurred due to suboptimal command values... undershooting or overshooting the 0.5 m target radius." **Takeaway: syntactic validity is solved; correct multi-step execution is not.** Don't cite "40%" as command quality, and don't cite "100%" as task success.

1.3 Existing LLM↔drone integrations you could reuse (skeptical maturity check)

Bottom line: every drone-specific MCP/agent repo today is an immature, mostly single-author POC. Read them as reference; depend on the underlying libraries (MAVSDK/pymavlink/MAVROS).

Project	Stack	Maturity (2026-06-08)	Verdict
PeterJBurke/droneserver (paper arXiv 2601.15486) ✓	MCP server ↔ MAVLink , wraps ~40-45 MAVSDK tools , tested in PX4 SITL across GPT/Claude/Gemini/local	4★, ~160 commits, v1.4.0, active but single-author, created Nov 2025	Best MCP reference ; implements the "Active Flight Management" fix (\$6)
ion-g-ion/MAVLinkMCP	MCP ↔ MAVLink/PX4	~19★, POC	Reference only
Bilalileri/EchoPilot	Voice→MCP→MAVSDK→PX4, Ollama, LangGraph	~19★, early-stage	Reference for the voice path
SSuperMK15/MAVLink-AI-Agent	Voice→Cohere+RAG→pymavlink	2★, 1 commit	MVP-scope gauge
nasa-jpl/rosa	LangChain ReAct agent for ROS, built-in safety	~1,540★, maintained	Only mature agent framework; ROS 2 only

1.4 LeRAAT — the closest *advisory* blueprint ✓ (but evidence is thin)

Aspect	Finding
Architecture	XPPython3 plugin in X-Plane 12 reads live params + ECAM warnings → ZeroMQ (TCP 5555) → RelayServer (RAG + LLM) → on-screen/VR overlay GUI . State machine Armed → Active → Interactive .
RAG	Real : FAISS + OpenAI text-embedding-3 + LangChain ensemble retriever, top-10 chunks. Corpus: A320 AFM, FAA directives, SOPs. (<i>No manuals ship — copyright.</i>)
LLM	GPT-4o (reasoning) + GPT-4o-mini (shorten for cockpit). Swappable backend.
Trigger	Master warning/caution event, pilot query button, or interactive mode.
Evaluation ⚠	Proof-of-concept only: 3 subject-matter experts, 2 scenarios, qualitative. "Resolved faster" is anecdotal, not measured. No accuracy/latency/hallucination metrics. Treat the "it helps pilots" claim as untested.
Code	github.com/sisl/LeRAAT, MIT, small (~9★). Ships mock_xp_plugin.py to run the server without X-Plane .
Stated gaps	NOTAM integration, voice interaction, hallucination risk — all "future work." Read-only/advisory; does not actuate .

Adapt to a UAV GCS: swap the X-Plane plugin for a **MAVLink/MAVSDK listener on PX4 SITL**; map its warning trigger onto **MAVLink STATUSTEXT severity, SYS_STATUS sensor health, EKF/failsafe flags, mode/arming changes**; shift the corpus from A320 AFM to **PX4 docs + airframe params + your ConOps + UAS regulations**; make advisories **operator-facing**. The ZeroMQ→FAISS/LangChain→GPT-4o+mini→overlay spine ports over almost unchanged.

1.5 TypeFly / MiniSpec — the closest *command-layer* blueprint ✓ (well-measured)

Aspect	Finding
MiniSpec	A BNF-defined mini-DSL the LLM emits instead of Python: two-char skill tokens, bounded loops, conditionals, and a special probe (mid-execution LLM query). Example plan = 18 tokens vs 41 for Python .

Aspect	Finding
Why it's faster	(a) Fewer output tokens — and each output token is ~ 2800x costlier than an input token (their fitted model), so "long prompt, short output" wins. (b) Stream interpretation : a parser executes statements as they arrive, before generation finishes.
Measured ✓	Latency : MiniSpec alone ≤32% reduction; +streaming up to 62% ; absolute 1.10–1.60 s. Tokens : up to 42% reduction, avg 34%. Success : 10/11 tasks 100%, obstacle-avoidance 70% (10 runs each).
Hardware / vision / LLM	Real DJI Tello ; edge server RTX 4090 ; YOLOv8 → text {object, bbox} list → cloud LLM; GPT-4 only.
Code	github.com/typefly/TypeFly, GPL-3.0, ~104★, active. Virtual cv2.VideoCapture mode runs without a drone; RobotWrapper for new platforms.
Limits	Poor geometric reasoning, no memory, re-sends most of the prompt each call.

Transferable idea: don't have the LLM emit verbose JSON/Python — define a **compact, short-verb command DSL/tool schema** over MAVLink (arm, takeoff, goto, rtl, loiter, land, + a probe-style state query). Fewer tokens = lower latency, and a **constrained grammar makes invalid/unsafe commands much harder to emit** (a real safety win). Use streaming/incremental dispatch — but gate every command behind the validator before it actuates.

1.6 Landscape / survey resources (for related-work & baselines)

- **UAVs_Meet_LLMs** — github.com/Hub-Tian/UAVs_Meet_LLMs (492★, *Information Fusion* 2025). Curated taxonomy (perception / VLN / planning / flight control / infrastructure). Key entries: TypeFly, **Tazir "From Words to Flight" (PX4/Gazebo)**, PromptCraft, LEVIOSA, SPINE, REAL, and **Tang et al. physical-safety benchmark**. ⚠ Content effectively **frozen ~March 2025** — not tracking 2025–26 work. Note: **no entry on MAVLink telemetry-log analysis** (a gap = your opening).
- **"Recent Advances in Transformer & LLMs for UAV Applications"** — Kheddar et al., arXiv **2508.11834** (Aug 2025, 39 pp, 300+ refs). Its simulator table flags **Gazebo+PX4 as the canonical control-research sim** (validates your platform). Open challenges it names: **latency, fragmented evaluation / no standard benchmarks, real-time embedded deployment, data scarcity**. Provides a **RAG hallucination-mitigation framework** and a concrete metric menu (BLEU / Cosine / Flesch / Gunning-Fog for NL report quality; plus proposed Hallucination Rate, Trajectory Success Reward, Edge-Latency-Per-Token).

2. Gaps & open problems

1. **Correct multi-step execution, not validity** ✓ — validity is ~100%; end-to-end mission success is ~40%. The gap (value correctness, perception, sequencing) is the real problem.
2. **Hallucination in safety-critical control** ✓ — universal agreement: never give the LLM full autonomous control.
3. **Latency** ✓ — ~1.1–5 s per inference. Fine for deliberation, fatal for fast loops. (TypeFly's DSL+streaming is the best demonstrated mitigation.)
4. **Grounding** ✓ — needs an explicit affordance/validation layer (SayCan-style).
5. **Fragmented evaluation / no standard benchmark** ✓ — named an open need by the survey; nobody has a standard UAV verification suite.
6. **Regulatory/airspace knowledge** ⚠ (**genuine gap**) — LeRAAT itself lists NOTAM integration as future work; no source grounds advice in authoritative airspace data.

7. **Telemetry-log analysis** ⚠️ (**genuine gap**) — the curated survey has no MAVLink-log-analysis entry; under-served vs. command-and-control.
-

3. What you could add (novelty / differentiators)

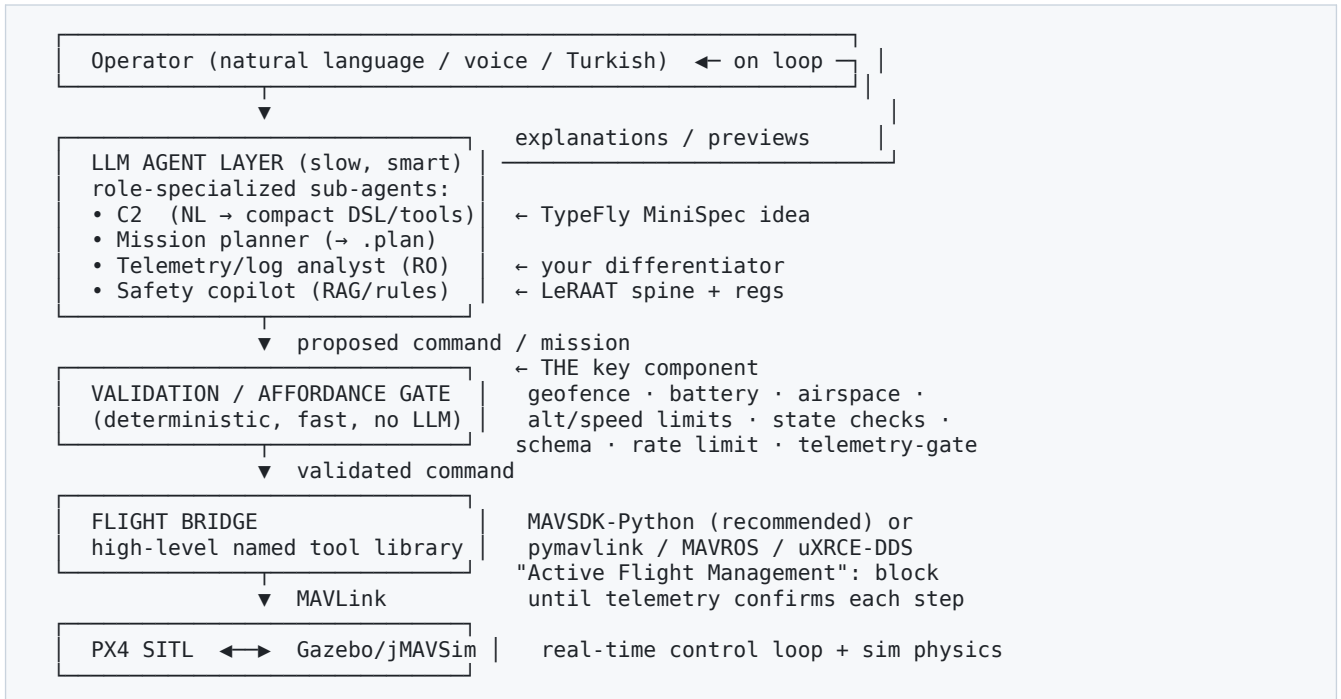
Each maps to a gap and is defensible for a thesis/competition/product:

1. **Telemetry & flight-log analysis / anomaly narration** ⚙️ — *lowest-risk, highest-novelty, read-only*. Train/eval on real **PX4 Flight Review** logs + **UAV-SEAD/ALFA** (§8). Auto-label anomalies from PX4's own logged events/failsafes; generate (telemetry-window → natural-language explanation) pairs.
 2. **Regulatory-grounded safety copilot** ⚙️ — RAG over CAA/SHGM rules, airspace classes, NOTAMs, manufacturer limits, so safety advice is *grounded*, not hallucinated. Fills LeRAAT's stated NOTAM gap.
 3. **Turkish-language, RAG-grounded operator assistant** ⚙️ — directly mirrors Baykar's public LLM/RAG hiring and the TEKNOFEST Turkish-NLP competition; optionally built on / benchmarked against **T3 AI** (the Turkish open LLM) (§10).
 4. **A UAV verification/benchmark suite** ⚙️ — report the *right* metrics: valid-command rate **and** mission success **and** unsafe-command rejection, geofence-violation catch, replanning correctness, hallucination-catch. The field wants this.
 5. **A compact, validated command DSL (MiniSpec-style) for MAVLink** ⚙️ — latency + safety win, with measured before/after numbers (a clean, citable contribution).
 6. **Embedded-latency study** ⚙️ — published "local model" numbers were desktop-GPU; a rigorous Jetson/RPi quantized benchmark is a real contribution and settles your deployment question.
-

4. Recommended architecture & tech stack

The golden rule (✓ universal across sources)

The LLM is an advisor/translator above a deterministic safety layer — never the controller. PX4 owns the real-time loop and failsafes. The LLM emits *intents*; the validator turns approved intents into MAVLink/MAVSDK calls.



Connector choice ⚠️⚙️ (engineering judgment; verified systems split between ROS 2 and Python-MAVLink middleware)

Connector	Use it when	Notes
MAVSDK-Python ★	Default. Start here.	High-level async API maps cleanly to LLM tool calls. PX4-first. What <code>droneserver</code> wraps.
pymavlink	You need a raw MAVLink message MAVSDK lacks	Lower-level; what MAVLink-AI-Agent uses.
MAVROS	You commit to ROS 2 (and want ROSA)	MAVLink↔ROS gateway.
px4_ros_com + uXRCE-DDS	Native ROS 2, low-latency uORB	PX4's recommended ROS 2 path; heavier.

MCP vs. plain function-calling ⚙️

For SITL-first, single-app: **start with native function/tool-calling against MAVSDK-Python**. Promote to a thin **FastMCP** server (model it on `PeterJBurke/droneserver`) only when you want multi-client reuse. Don't fork an immature drone-MCP repo.

How the four roles map ⚙️

- **C2** → tool/DSL agent over the flight bridge; every call through the validation gate.
- **Mission planning** → emit **QGC .plan JSON**; upload via MAVSDK mission API; replan on telemetry triggers.
- **Telemetry/log analysis** → **read-only** agent over MAVLink streams + `.uLog` (parse with `pyulog`). *Build first.*
- **Safety copilot** → LeRAAT-style RAG over geofence + regulatory data; acts as the affordance reasoner + operator advisor.

5. Deployment trade-offs (cloud vs onboard vs hybrid)

✓ Literature recommends **hybrid**: latency-critical tasks onboard, heavy reasoning to the cloud.

Option	Pros	Cons
Cloud frontier API (Claude/GPT)	Most capable; best for planning, log analysis, safety reasoning	Needs connectivity; ~5 s latency; offline = dead
Onboard local/quantized (Jetson/RPi)	Offline; data stays local	Limited; ~40% mission success in practice; ⚠ published "local model" latency was on a desktop RTX 4090/4070, NOT a Jetson — embedded latency will be much worse
Hybrid ★	Local fast/safety path + cloud deliberation	More complex; needs a clean split point

⚙ **For SITL development this is moot — run on your dev box / cloud API.** Defer the embedded decision; then benchmark quantized models (incl. **T3 AI**) on your actual target board — that benchmark is itself a contribution (§3.6).

6. Safety & guardrails (✓ mandatory, convergent across all sources)

1. **LLM out of the hard real-time loop** — always. PX4 owns flight control + failsafes.
2. **Deterministic validation gate** before any command — schema, geofence, alt/speed limits, battery/state preconditions, rate limiting. Reject + explain on violation.
3. **Human on the loop** — operator approves/overrides; **command preview/diff before execution**.
4. **Fail-safe thresholds** — uncertain or out-of-bounds output → **require human intervention**.
5. **Redundant double-checking** + an **automated validation loop** that detects bad commands and feeds them back to regenerate, then a **human stage-gate**.
6. **Lean on PX4's own safety net** — geofence, RTL, battery failsafe, kill switch remain authoritative below the LLM.

⚠ *The sequential-command trap (verified in `droneserver` / arXiv 2601.15486 — internalize this)*
Naive "MAVSDK methods as tools" **fails**: the LLM "may send the take off and fly to command in rapid succession, and the drone would not have time to ascend... resulting in a low altitude flight towards the final destination... resulting in a crash," and "would immediately assume that the drone was at its new location after sending a goto command." **The fix** (`droneserver`'s "Active Flight Management"): `takeoff()` doesn't return until target altitude is reached; `land()` is gated until at destination; auto-land waits for ground contact (checked every 2 s). The bridge becomes "a kind of ground control station with its own internal memory and logic." **This telemetry-gating is the real engineering work — budget for it.**

7. Evaluation (SITL-first)

- ✓ Simulation testing is the gate before HITL/hardware. Report **both** metric families (the §1.2 lesson):
- **Valid-command rate** (syntactic/structural) — easy; expect ~100% from decent models.
 - **Mission success rate** (end-to-end task completion) — the honest headline; baseline ~40%.
 - **Unsafe-command rejection rate** of the validation gate; **geofence-violation catch rate**.
 - **Hallucination-catch rate** of the validation loop; **replanning correctness** under injected faults (sensor loss, low battery, no-fly intrusion).

- **Latency / Edge-Latency-Per-Token**; for NL report/summary quality use **BLEU**, **cosine similarity**, **Flesch Reading Ease**, **Gunning-Fog** (per survey 2508.11834).
- **Safety eval harness**: reuse **Tang et al.'s physical-safety benchmark** (HF TrustSafeAI/llm_physical_safety_benchmark, arXiv 2411.02317).

⚙️ Use an **AutoSimTest-style scenario generator** to auto-produce test missions; fault-inject in PX4 SITL; only then move to HITL.

8. Datasets

✓ all verified to exist. ⚠️ flags below are load-bearing — read them before committing.

Telemetry / flight-log (best fit — UAV/PX4 native)

Dataset	Size	Format / License	Role & caveats
PX4 Flight Review (logs.px4.io/browse) ✓	~ 123k ULog files (2024, third-party estimate)	ULog · CC-BY 4.0 · bulk download via <code>PX4/flight_review/app/download_logs.py</code> , parse with <code>pyulog</code>	Telemetry-explanation / anomaly-narration. ⚠️ Unlabeled — engineer labels from logged events/failsafes. Heterogeneous airframes/firmware → schema drift.
UAV-SEAD (arXiv 2602.13900) ✓	1,396 real PX4 logs , > 52 h , normal + anomalous	labeled	Strongest direct PX4 anomaly match — same ecosystem.
ALFA (arXiv 1907.06268) ✓	47 flights, 8 fault types	MAVROS/MAVLink rosbags , labeled	Clean fault ground-truth.
BASiC (via survey 2508.11834) ✓	GPS/RC/accel/gyro/compass/baro → 17 features	sensor-failure prediction	Closest to MAVLink telemetry feature engineering.

Aviation QA (for RAG/safety-copilot — but crewed-aviation, not UAV)

Dataset	Size	Format / License	Role & caveats
AviationQA (sakharang/AviationQA) ✓	~ 1.08M QA pairs	CSV (id/Q/A) · CC-BY 4.0	Template-generated from 12k NTSB reports (NTSB-only, not ASRS). Good for vocab/volume; weak for reasoning (repetitive templates).
Aviation_QA (Timilehin674/Aviation_QA) ✓	~350k (author claim; ⚠️ unverified — viewer broken)	SQuAD v1.1 JSON · ⚠️ LICENSE CONFLICT: YAML says MIT, README says CC-BY-4.0 — resolve before commercial/competition use	NTSB+ASRS, LLaMA-3.3-generated + human-verified. Better for RAG/extractive eval (context + answer spans + FAA ISAM categories).
Physical-Safety Benchmark (Tang et al., TrustSafeAI/...) ✓	—	HF dataset	Safety-advisory evaluation , not training.

⚠️ **Domain-mismatch caveat**: AviationQA / Aviation_QA are **crewed aviation** — they teach safety vocabulary, not UAV/MAVLink/PX4 specifics. Pair with PX4 docs + the log corpus for the UAV layer.

9. Concrete starting setup & first milestones

Environment ⚙️: Ubuntu 22.04 (or Docker) · **PX4-Autopilot SITL** + **Gazebo** (jMAVSim for speed) · **QGroundControl** · **MAVSDK-Python** flight bridge (pymavlink/pyulog for raw msgs & logs) · LLM via API during dev. Python agent layer (native tool-calling first; LangGraph if you want explicit graph orchestration).

Milestones (lowest-risk → highest):

1. **PX4 SITL + MAVSDK "hello world"** — arm/takeoff/goto/land from a script. No LLM.
2. **Read-only telemetry/log analyst (role 3)** — summarize live telemetry + parse `.ulog`; pre-flight checklist. *Safe first win + your novelty.*
3. **NL command & control (role 1)** — compact DSL + **validation gate + telemetry-gated execution** (solve the \$6 trap here).
4. **Mission planning (role 2)** — NL → QGC `.plan` → upload → fly in SITL; add replanning triggers.
5. **Safety copilot (role 4)** — RAG over geofence + regulatory data (LeRAAT spine); confidence-gated escalation.
6. **Evaluation harness** — scenario generation + the \$7 metrics.
7. **(Later)** HITL on sim boards; embedded-latency benchmark to settle deployment.

10. Baykar & TEKNOFEST positioning (strategic context)

✓ *The ecosystem is one chain: Selçuk Bayraktar founded & chairs the **T3 Foundation**; Baykar is T3's sole corporate supporter; T3 runs **TEKNOFEST**; TEKNOFEST runs both competitions. Aligning with TEKNOFEST is aligning with the Baykar orbit.*

Why this matters for your project (✓ verified signals)

- **Baykar publicly hired an LLM/RAG engineer** (GPT-3+/LLaMA/Falcon, **fine-tuning + RAG**) — your project's exact skill set.
- **T3 AI** — an **open-source Turkish LLM** by T3 + Baykar + Cezeri (beta Jul 2025). → a **Turkish base model to build on / benchmark against**, rather than a US model.
- Baykar's in-house **Central Command Control** does **multi-UAV fleet supervision** (MYGS, Air Traffic Coordination, mission planning) but its public material shows **no natural-language / AI interface** — a visible gap your assistant speaks to.
- ⚠️ Could **not** verify Baykar ships an LLM inside a flight/operator product today — treat as an **open opportunity**, not an existing product.

Competition fit (honest assessment)

Competition	Fit	How to position
Türkçe Doğal Dil İşleme Yarışması (Turkish NLP) ✓	STRONG — primary venue	Enter the Free Category with Turkish NL drone command & control + Turkish aviation RAG/QA . The 2025 Scenario Category ("autonomous call center") was essentially RAG + agentic actions in Turkish — same architecture. Prize pool 600k TL.

Competition	Fit	How to position
Savaşan İHA (combat UAV, Baykar-led) 	WEAK/INDIRECT	It's vision + autonomy (Lidar/Radar banned for locking) — an LLM is too slow for the scored core. Do not pitch the LLM as the combat differentiator. Off-path byproducts only: spec-RAG for the design report, post-flight log triage/debrief , ground-side config queries.

Positioning suggestions (informed inference — *not* stated Baykar requirements)

1. **Frame as a Turkish, RAG-grounded flight assistant**, built on/benchmarked vs **T3 AI** — mirrors Baykar's hiring + model investment. (*Highest-leverage move.*)
2. **NL supervisory layer for multi-UAV ops** — target the Central Command Control gap ("show me the drone lowest on fuel," "summarize alarms across the fleet," "re-plan UAV-3 around weather").
3. **Operator workload reduction** for single-operator/many-drone (swarm is coming — K2 swarm tests are public).
4. **Log triage / debrief in Turkish** — the most defensible byproduct that also helps a Savaşan İHA team.
5. **Decision-support, human-in-the-loop framing** for a defense audience — surfaces info and proposes operator-confirmed commands; avoids overclaiming autonomous weapons control.

11. AI-assisted coding tooling for *building* this project

 *Engineering judgment.*  -available = confirmed wired into the current environment.

MCP servers

- `context7`  -available — **most useful**. Current docs for PX4, MAVSDK, pymavlink, MAVROS, ROS 2 on demand; keeps generated code correct as APIs drift.
- `playwright`  -available — browser automation for any web GCS/dashboard you build + UI testing.
- `ros-mcp-server`  — only if you go ROS 2.
- **A thin MAVSDK MCP**  — optional/later (model on `droneserver`); only for multi-client reuse.

Claude Code skills (available here)

- `brainstorming` — *before* committing to architecture. **Recommended next step.**
- `writing-plans / executing-plans` — turn the \$9 milestones into a staged, checkpointed plan.
- `test-driven-development` — the validation gate + command logic are safety-relevant; tests first.
- `systematic-debugging` — for SITL/MAVLink integration bugs.
- `mcp-builder` — if/when you wrap the flight bridge as MCP.
- `webapp-testing / frontend-design` — for an operator dashboard / GCS web UI.

Sources (primary, fact-checked)

Foundational: Microsoft *ChatGPT for Robotics* · github.com/microsoft/PromptCraft-Robotics · SayCan say-can.github.io (arXiv 2204.01691) · *Code as Policies* arXiv 2209.07753

Direct UAV/PX4: *Tazir From Words to Flight* (WCSE 2023) · *Taking Flight with Dialogue* arXiv 2506.07509 + github.com/limshoonkit/ros2-agent-ws · **LeRAAT** arXiv 2503.16477 + github.com/sisl/LeRAAT · **TypeFly** arXiv 2312.14950 + github.com/typefly/TypeFly (IEEE TMM 2025) · *Universal LLM-Drone C2 / droneserver* arXiv 2601.15486 + github.com/PeterJBurke/droneserver · github.com/SuperMK15/MAVLink-AI-Agent

Surveys/landscape: github.com/Hub-Tian/UAVs_Meet_LLMs (*Information Fusion* 2025) · Kheddar et al. arXiv 2508.11834

Datasets: logs.px4.io/browse (CC-BY) + [PX4/flight_review](https://logs.px4.io/flight_review) + [PX4/pyulog](https://logs.px4.io/pyulog) · UAV-SEAD arXiv 2602.13900 · ALFA arXiv 1907.06268 · huggingface.co/datasets/sakharamg/AviationQA · huggingface.co/datasets/Timilehin674/Aviation_QA · Tang et al. *physical-safety* arXiv 2411.02317

Libraries: github.com/mavlink/MAVSDK-Python · github.com/ArduPilot/pymavlink · github.com/mavlink/mavros · github.com/PX4/px4_ros_com · docs.px4.io/main/en/ros2/

Baykar/TEKNOFEST: teknofest.org (Turkish NLP & Savaşan İHA pages) · t3vakfi.org/tr/projelerimiz/t3-ai · Baykar Central Command Control (baykartech.com)

Refuted in v1 fact-checking (do not cite): "10⁵× slower than YOLO" latency ratio; the exact "PX4 v1.14.3 + Gazebo Harmonic + ROS 2 Humble + LangGraph ReAct" stack config.

Corrected in v2: the "40% mission success" figure is *not* a command-quality metric — see §1.2.